

Raport z negocjacji

Daniel Szeląg
29 marca 2026

1 Wstęp.....	3
2 Ogólny opis projektu.....	3
2.1 Działanie aplikacji.....	3
2.2 Wstępny wybór technologii.....	3
2.3 Założenia i ograniczenia.....	3
3 Opis aplikacji.....	4
3.1 Spis widoków.....	4
3.2 Mechanizmy oraz logika biznesowa.....	5
3.3 Zabezpieczenia i obsługa błędów.....	6
4 Słowniczek pomocniczy.....	6

1 Wstęp

W dniu 29.03.2026 r. odbyło się drugie spotkanie negocjacyjne dotyczące stworzenia aplikacji desktopowej służącej do automatycznego sprawdzania kolokwiiów studenckich z programowania. Podczas spotkania omówiono wstępne wymagania funkcjonalne, oczekiwania co do interfejsu oraz preferowane technologie. Niniejszy raport stanowi podsumowanie ustaleń z tego spotkania, uwzględniając ustalenia poczynione po pierwszej iteracji negocjacji.

2 Ogólny opis projektu

2.1 Działanie aplikacji

Będzie to aplikacja okienkowa służąca do automatycznego sprawdzania kolokwiiów studenckich z programowania, wspomagająca pracę wykładowcy. Po uruchomieniu programu użytkownik przeciąga pliki z rozwiązaniami studentów (pliki .cs) na wyznaczone pole w oknie aplikacji. Aplikacja sama sprawdza każde rozwiązanie, porównując wyniki z poprawnymi odpowiedziami zapisanymi w pliku XLSX. Za każde poprawne rozwiązanie przyznawane są punkty. Wyniki sprawdzenia wyświetlane są w tabeli, a na żądanie użytkownika mogą zostać wyeksportowane do pliku XLSX.

2.2 Wstępny wybór technologii

Językiem programowania wybranym do realizacji projektu jest C# z wykorzystaniem technologii Windows Forms. Do dynamicznego kompilowania plików źródłowych .cs zastosowany zostanie kompilator Roslyn (Microsoft.CodeAnalysis.CSharp), a do ładowania i uruchamiania skompilowanych assembly zastosowany zostanie mechanizm Assembly.LoadFrom z przestrzeni nazw System.Reflection. Klucz odpowiedzi przechowywany będzie w plikach XLSX (arkusze kalkulacyjne Excel). Ponadto w celu sprawnego zarządzania projektem wewnątrz zespołu zastosowany zostanie system kontroli wersji Git.

2.3 Założenia i ograniczenia

Założenia:

- Aplikacja uruchamiana jest na komputerze z systemem Windows 10 lub nowszym
- Dane przechowywane są lokalnie, archiwizacja odbywa się poprzez wygenerowane pliki XLSX
- Prace studenckie dostarczane są w formie plików źródłowych .cs zgodnych z platformą .NET
- Struktura kodu w plikach .cs jest dowolna (nie jest wymagana konkretna konwencja nazewnictwa klas czy metod)
- Klient wskazuje pliki poprzez mechanizm drag & drop (przeciągnięcie plików na wyznaczone pole w oknie aplikacji)
- Plik XLSX z odpowiedziami zawiera osobne zakładki dla każdego zadania (np. „Zadanie 1”, „Zadanie 1a”, „Zadanie 2” itd.)
- Każda zakładka w pliku XLSX zawiera kolumny: Zadanie, Parametry, Odpowiedź
- Obsługiwany jest jeden zestaw zadań (bez podziału na grupy A/B)
- Za każde poprawne rozwiązanie przyznawane są punkty zgodnie z kluczem odpowiedzi
- Wyniki prezentowane są w formie zbiorczego zestawienia
- Aplikacja umożliwia eksport wyników do pliku XLSX
- Aplikacja zawiera mechanizm zabezpieczenia przed nieskończoną pętlą w kodzie studenckim (timeout)

Ograniczenia:

- Aplikacja działa wyłącznie na systemie Windows
- Brak trybu wielo-użytkownikowego, aplikacja obsługuje jednego klienta na jednym stanowisku
- Brak synchronizacji danych między komputerami
- Aplikacja nie sprawdza jakości kodu źródłowego, weryfikowane są wyłącznie wyniki działania skompilowanych programów

3 Opis aplikacji

Aplikacja będzie automatycznie sprawdzać kolokwia studenckie z programowania, wspomagając pracę wykładowcy. Po uruchomieniu programu klient zobaczy główne okno z polem do przeciągania plików .cs oraz możliwością wskazania pliku XLSX z kluczem odpowiedzi. Po załadowaniu plików aplikacja skompiluje każdą pracę, uruchomi ją z danymi testowymi i porówna wyniki z oczekiwanymi odpowiedziami. Klient będzie mógł przeglądać wyniki sprawdzenia w tabeli oraz eksportować zestawienie do pliku XLSX.

3.1 Spis widoków

Interfejs graficzny aplikacji powinien być prosty i przejrzysty. Składać się będzie z jednego głównego okna podzielonego na następujące sekcje:

Pole drag & drop: obszar do przeciągania plików .cs z rozwiązaniami studentów

Przycisk wyboru pliku XLSX: pozwala wskazać plik z kluczem odpowiedzi

Przycisk [Sprawdź]: uruchamia proces kompilacji i weryfikacji

Pasek informacyjny: wyświetla liczbę załadowanych plików oraz nazwę wybranego pliku XLSX

Tabela wyników: prezentuje zbiorcze zestawienie: Student, Zadanie, Oczekiwany wynik, Uzyskany wynik, Punkty, Status

Przycisk [Eksportuj do XLSX]: zapisuje wyniki do pliku arkusza kalkulacyjnego

The mockup shows a window titled "Sprawdzanie Kolokwiów" with standard Windows window controls. The main area is divided into several sections:

- Drag & Drop Area:** A large box with the text "Przeciągnij pliki .cs tutaj".
- XLSX Selection:** A button labeled "Wybierz plik XLSX" and a status bar below it showing "Załadowane pliki: 0 | XLSX: (brak)".
- Check Button:** A green button labeled "Sprawdź".
- Filters and Export:** At the bottom, there are date pickers for "Od: 4/ 1/2026" and "Do: 4/30/2026", a blue "Filtruj" button, and a red "Eksportuj do XLSX" button.
- Status Bar:** The bottom left corner shows "Gotowy" and the bottom right corner has a standard Windows taskbar icon.

Przykładowy wygląd aplikacji

3.2 Mechanizmy oraz logika biznesowa

Aplikacja sprawdza prace studenckie na podstawie klucza odpowiedzi. Poniżej opisano główne mechanizmy:

Ładowanie plików źródłowych:

Klient przeciąga pliki .cs na wyznaczone pole w oknie aplikacji (drag & drop). Aplikacja odczytuje ścieżki upuszczonych plików i wyświetla listę załadowanych prac wraz z liczbą plików. Nazwa pliku służy jako identyfikator studenta (np. KowalskiJan_Zadanie1.cs).

Plik XLSX z kluczem odpowiedzi:

Plik XLSX zawiera klucz odpowiedzi w formie arkusza kalkulacyjnego z osobnymi zakładkami dla każdego zadania (np. „Zadanie 1a”, „Zadanie 1b”, „Zadanie 1c” itd.). Każda zakładka zawiera dane w formacie:

	A	B	C
1	Zadanie	Parametry	Odpowiedź
2	Zadanie 1a	5, 10	15
3	Zadanie 1b	3, 7	10
4	Zadanie 1c	-1, 1	0

Nazwa zadania w zakładce odpowiada fragmentowi nazwy pliku studenta, dzięki czemu aplikacja automatycznie dopasowuje pracę studenta do odpowiedniego zestawu testów.

Kompilacja i uruchomienie:

Dla każdego załadowanego pliku .cs aplikacja:

kompiluje plik źródłowy do assembly w pamięci za pomocą kompilatora Roslyn (Microsoft.CodeAnalysis.CSharp),

w przypadku błędów kompilacji zapisuje informację o błędzie i przyznaje 0 punktów za dane zadanie,

ładuje skompilowane assembly za pomocą Assembly.LoadFrom(),

wyszukuje odpowiednią klasę i metodę za pomocą mechanizmu refleksji (System.Reflection), struktura kodu jest dowolna, aplikacja dynamicznie odnajduje punkt wejścia,

wywołuje metodę z parametrami z pliku XLSX,

porównuje zwrócony wynik z oczekiwaną odpowiedzią,

przyznaje punkty za poprawne rozwiązanie.

Eksport danych:

Po kliknięciu przycisku „Eksportuj do XLSX” aplikacja generuje plik arkusza kalkulacyjnego zawierający zbiorcze wyniki sprawdzenia. Plik zawiera kolumny: Student, Zadanie, Oczekiwany wynik, Uzyskany wynik, Punkty, Status. Użytkownik wybiera lokalizację zapisu pliku przez standardowe okno dialogowe zapisu. Wygenerowany plik XLSX stanowi jednocześnie formę archiwizacji wyników — nie jest wymagana zewnętrzna baza danych.

Przykładowe zastosowanie:

Wykładowca uruchamia aplikację i przeciąga 30 plików .cs od studentów na pole drag & drop. Następnie wskazuje plik odpowiedzi_kolokwium1.xlsx zawierający klucz odpowiedzi z zakładkami dla 5 zadań (każde po 4 punkty, łącznie 20 punktów). Aplikacja kompiluje kolejno każdy plik, uruchamia 5 testów i porównuje wyniki. Po chwili wykładowca otrzymuje pełne zestawienie, np. studentka Anna Nowak uzyskała 16/20 punktów. Na koniec eksportuje wyniki do pliku wyniki_kolokwium1.xlsx, który wykorzystuje do wystawienia ocen.

3.3 Zabezpieczenia i obsługa błędów

Zabezpieczenie przed nieskończoną pętlą (timeout):

Każde skompilowane assembly uruchamiane jest z ograniczonym czasem wykonania (timeout). Jeżeli kod studencki nie zakończy działania w wyznaczonym czasie, jego wykonanie zostaje przerwane, a w wyniku odnotowywany jest błąd przekroczenia czasu. Mechanizm ten zapobiega zawieszeniu całej aplikacji w przypadku nieskończonych pętli lub nadmiernie złożonych obliczeń w kodzie studenckim.

Izolacja wykonania:

Dynamiczne kompilowanie i ładowanie nieznanych plików .cs niesie ryzyko wykonania złośliwego kodu. Należy rozważyć uruchamianie assembly w izolowanym środowisku (np. osobna AppDomain lub proces z ograniczonymi uprawnieniami).

Obsługa wyjątków:

Aplikacja przechwytuje wyjątki rzucane przez kod studencki i raportuje je jako błędy bez przerywania procesu sprawdzania pozostałych prac. Dzięki temu jeden wadliwy plik nie blokuje weryfikacji całego zestawu.

4 Słowniczek pomocniczy

GUI: graficzny interfejs użytkownika (Graphical User Interface)

C#: wieloparadygmatowy język programowania firmy Microsoft

Assembly: skompilowana jednostka kodu .NET (plik DLL lub EXE) zawierająca kod pośredni (IL)

Refleksja (Reflection): mechanizm .NET umożliwiający dynamiczne badanie i wywoływanie typów, metod i właściwości w czasie wykonania

Assembly.LoadFrom: metoda klasy System.Reflection.Assembly służąca do dynamicznego ładowania assembly z podanej ścieżki pliku

Roslyn: platforma kompilatorowa .NET umożliwiająca m.in. programową kompilację kodu C#

Git: rozproszony system kontroli wersji

XLSX: format pliku arkusza kalkulacyjnego Microsoft Excel (Office Open XML Spreadsheet)

Drag & drop: mechanizm przeciągania i upuszczania plików na okno aplikacji

Timeout: mechanizm ograniczenia czasu wykonania operacji, po przekroczeniu wyznaczonego limitu operacja jest przerywana